

第9章: スタイリングとUI (NativeWind)

アプリの機能は完成しました。この章では「見た目」をより効率よく整える方法を学びます。本書では **NativeWind** (Tailwind CSSの書き方をReact Nativeで使えるライブラリ) を導入します。まずスタイリング手法の選択肢を比較し、NativeWindを導入して、これまで作った画面をより少ないコードで書き直します。

1. スタイリング手法の選択肢を比較する

第4章では **StyleSheet** を使いました。React Nativeのスタイリングには他にも選択肢があります。「どんな場合にどれを選ぶべきか」を解説します。

1.1 比較表

手法	記述スタイル	記述量	学習コスト	Web版との共通性	向いている人
StyleSheet (RN標準)	オブジェクトで定義	多い	低 (追加不要)	△	追加ライブラリを増やしたくない人
NativeWind (本書採用)	クラス名で指定	少ない	低 (Tailwind経験者は即)	◎	Web/TailwindとUIを共通化したい人
Tamagui	専用コンポーネント	中	中～高	△	高性能・高機能を求める人
UIキット (gluestack / React Native Paper等)	完成部品を使う	少ない	中	△	デザインを自分で考えず素早く作りたい人

1.2 それぞれの解説

◆ StyleSheet (RN標準)

第4章で使った方法。 `StyleSheet.create({ ... })` でスタイルを定義し、名前で参照します。

- **長所:** 追加ライブラリ不要／React Nativeの基本そのもの／どんな教材でも通用する
- **短所:** 記述量が多い／小さな調整でもスタイル定義が増えていく

◆ NativeWind（本書の採用）

Tailwind CSS という「短いクラス名で見た目を指定する」仕組みを、React Nativeで使えるようにしたもの。

`className="text-lg font-bold text-blue-600"` のように書きます。

- **長所:** 記述量が少ない／Web版チュートリアル（Tailwind）と書き方が同じ／デザインの一貫性を保ちやすい
- **短所:** クラス名を覚える必要がある（が、よく使うものは限られる）

◆ Tamagui（タマグイ）

高性能を売りにした高機能スタイリング＋UIライブラリ。アニメーションやテーマ切替に強い。

- **長所:** 高性能／機能豊富
- **短所:** 学習コストが高め／初心者にはオーバースペックなことが多い

◆ UIキット（gluestack-ui / React Native Paper など）

ボタンやカードなどの「完成された部品」が最初から用意されているライブラリ。

- **長所:** デザインを考えなくても整った見た目になる／開発が速い
- **短所:** キット独自の作法を学ぶ／細かいカスタマイズに制約

どんな時にどれを選ぶ？（まとめ） - Web/Tailwindの経験があり、UIを共通化したい → NativeWind（本書） - ライブラリを増やさず基本に忠実でいたい → StyleSheet - 凝ったアニメーションや高性能が必要 → Tamagui - デザインを自分で作らず最速で形にしたい → UIキット

2. NativeWind のセットアップ

2.1 インストール

第1章で作った `my-books-app` フォルダで、次を順に実行します。

```
npx expo install nativewind tailwindcss react-native-reanimated react-native-safe-area-context
# nativewind                : 本体（TailwindをRNで使えるようにする）
# tailwindcss                : Tailwindのクラス定義の土台
# react-native-reanimated    : NativeWindが内部で利用するアニメーション部品
# react-native-safe-area-context: 安全領域の部品（第4章で登場。NativeWindと相性よく使う）
```

2.2 Tailwind設定ファイルを作る

次のコマンドで、Tailwindの設定ファイルのひな形を生成します。

```
npx tailwindcss init
# tailwindcss init : tailwind.config.js という設定ファイルを生成するコマンド
```

生成された `tailwind.config.js` を次のように書き換えます。

▼ このコードがやること（先に日本語で）：NativeWind（Tailwind）に「クラス名をどのファイルから探すか」と「NativeWind用の設定一式を読み込む」ことを教える設定ファイルです。一番大事なのは `content` ——ここに書いたファイルの中で実際に使われているクラスだけがスタイルとして取り込まれます。書き換える箇所は数行だけで、残りはほぼ決まり文句（おまじない）です。詳しい意味は各行のコメントを見てください。

```
// tailwind.config.js - Tailwind/NativeWindの設定

/** @type {import('tailwindcss').Config} */
module.exports = {
  // content : Tailwindのクラスを「どのファイルから探すか」を指定する
  // app配下とcomponents配下の .tsx などを対象にする
  content: ["../app/**/*.{js,jsx,ts,tsx}", "./components/**/*.{js,jsx,ts,tsx}"],
  // presets : NativeWind用の設定一式を読み込む（おまじない）
  presets: [require("nativewind/preset")],
  theme: {
    extend: {}, // ここに独自の色やサイズを追加できる（今は空でOK）
  },
  plugins: [],
};
```

content の指定が重要: Tailwindは「実際に使われているクラスだけ」を最終的に取り込みます。`content` で指定したファイルからクラス名を探すので、ここにファイルの場所を正しく書かないとスタイルが効きません。`./app/**/*.{...}` は「appフォルダ以下の全階層（**）の、指定拡張子のファイル」という意味です。

2.3 グローバルCSSを作る

`app` フォルダに `global.css` を新規作成し、Tailwindの基本指定を書きます。

▼ このコードがやること（先に日本語で）：Tailwindの土台となるスタイルを読み込むための、3行だけのCSSファイルを作ります。`@tailwind base / components / utilities` の3行を書くことで、`text-lg` や `bg-white` といった便利クラスがアプリで使えるようになります。これはNativeWindを使うための「お決まりの3行」なので、丸ごとそのまま書けばOKです。

```
/* app/global.css - Tailwindの基本スタイルを読み込む */
@tailwind base;          /* Tailwindの土台スタイル */
@tailwind components;    /* コンポーネント系スタイル */
@tailwind utilities;     /* text-lg などの便利クラス群 */
```

2.4 Babel と Metro の設定

NativeWindを動かすため、2つの設定ファイルを調整します。内部の仕組みなので「こう書く」と覚えれば十分ですが、**それぞれが何をしているのか**を知っておくと、エラー時に対処しやすくなります。

そもそも Babel と Metro とは？ あなたが書く `className="text-lg"` のようなコードは、そのままではスマホは理解できません。アプリを動かす前に、**スマホが分かる形へ変換（へんかん）** する道具が必要です。 - **Babel**（バベル） ... 新しい書き方（TypeScript、JSX、NativeWindのclassName）を、スマホが解釈できる素のJavaScriptへ**翻訳する**道具。 - **Metro**（メトロ） ... たくさんのファイルや部品を1つにまとめてスマホへ届ける道具（「バンドラ＝束ねるもの」と呼びます。第1章で登場）。

NativeWindは「`className` を本物のスタイルへ変換する」仕組みなので、この**BabelとMetroの両方に「NativeWindの変換も通してね」と教える**必要があります。それが下の2ファイルの役割です。

`babel.config.js`（無ければ作成）：

▼ このコードがやること（先に日本語で）：Babel（コードをスマホが分かる形へ翻訳する道具）に「`className`をスタイルに変える翻訳ルールも使ってね」と教える設定です。`jsxImportSource: "nativewind"` と `"nativewind/babel"` の2つが、その指定の中心です。仕組みは下のblockquoteで詳しく説明していますが、まずは丸ごとコピーで構いません。詳細は各行のコメントを参照してください。

```
// babel.config.js - コード変換の設定
module.exports = function (api) {
  api.cache(true); // 設定をキャッシュして高速化（おまじない）
  return {
    // presets : 変換ルールの一式。Expo用 + NativeWind用 (jsxImportSource) を指定
    presets: [
      ["babel-preset-expo", { jsxImportSource: "nativewind" }],
      "nativewind/babel",
    ],
  };
};
```

`metro.config.js`（無ければ作成）：

▼このコードがやること（先に日本語で）：Metro（ファイルを1つにまとめてスマホへ届ける道具）に「NativeWindの機能を足し、先ほど作った **global.css** を読み込ませる」設定です。

withNativeWind(...) は「Expoの標準設定を受け取って、NativeWind対応版にして返す」関数だと考えてください。これも基本はコピーでOK。詳しい役割は下のblockquoteと各行のコメントにあります。

```
// metro.config.js - バンドラ(Metro)の設定。global.cssをNativeWindに渡す
const { getDefaultConfig } = require("expo/metro-config");
const { withNativeWind } = require("nativewind/metro");

const config = getDefaultConfig(__dirname); // Expoの標準設定を取得

// withNativeWind : 標準設定にNativeWindの機能を足す。input に先ほどのCSSを指定
module.exports = withNativeWind(config, { input: "./app/global.css" });
```

2つの設定ファイルが何をしているか（要点）：- **babel.config.js** ... **babel-preset-expo**（Expo標準の変換ルール）に、**{ jsxImportSource: "nativewind" }** を渡しています。これは「JSX（**<View className= "... ">**のような画面の書き方）を変換するとき、NativeWindの仕組みを使ってね」という指定です。続く **"nativewind/babel"** も、classNameを解釈するための変換ルールです。つまりBabel側に「classNameをスタイルに変える翻訳ルール」を追加しています。- **metro.config.js** ... **getDefaultConfig** でExpoの標準設定を取得し、**withNativeWind(config, { input: "./app/global.css" })** で「標準設定にNativeWindの機能を足し、先ほど作った **global.css** を読み込ませる」設定にしています。

withNativeWind は「設定を受け取って、NativeWind対応版にして返す」関数だと考えてください。

難しく見えますが、やっていることは「BabelとMetroの両方に、NativeWindの変換を組み込む」だけです。丸ごとコピーで構いませんが、「className がスタイルに化けるのは、この2ファイルのおかげ」と覚えておくと、もしスタイルが効かないとき真っ先にここを見直せます。

2.5 グローバルCSSを読み込む

app/_layout.tsx（第6章で作成）の先頭に、CSSの読み込みを1行足します。

```
// app/_layout.tsx の一番上に追加
import "./global.css"; // NativeWindのスタイルをアプリ全体で有効にする（先頭で読み込む）

import { Stack } from "expo-router";
// （以下は第6章のまま）
```

2.6 設定を反映させる

設定ファイルを変えたときは、開発サーバーをキャッシュを消して再起動します。

```
npx expo start --clear
```

--clear : 以前のキャッシュ（一時保存データ）を消して起動する。設定変更を確実に反映させる

--clear を付ける理由: Metro（変換ツール）は速度のため変換結果を一時保存します。設定を変えた直後は古い保存が残っていて反映されないことがあるため、**--clear** で消してから起動します。「設定を変えたのに反映されない」ときの定番の対処です。

3. NativeWind の基本的な書き方

NativeWindでは、`style={...}` の代わりに `className="..."` にクラス名を並べて見た目を指定します。

▼ このコードがやること（先に日本語で）: NativeWindの基本である `className="..."` の書き方を、画面中央に文字を表示する小さな例で体験します。ポイントは「`className` の中に、見た目を表す短いクラス名を空白で区切って並べる」こと——例えば `flex-1`（画面いっぱい）、`justify-center`（縦中央）、`text-2xl`（文字大きめ）のようにです。1つ1つのクラスが何を意味するかは、コード内のコメントで確認してください。

```
import { View, Text } from "react-native";

export default function Demo() {
  return (
    // className に空白区切りでクラスを並べる
    <View className="flex-1 justify-center items-center bg-white">
      { /* flex-1 → flex:1 / justify-center → 縦中央 / items-center → 横中央 / bg-white →
      背景白 */ }
      <Text className="text-2xl font-bold text-slate-800">📖 書籍管理アプリ</Text>
      { /* text-2xl → 文字大きめ / font-bold → 太字 / text-slate-800 → 濃いグレー文字 */ }
    </View>
  );
}
```

3.1 第4章のStyleSheetとの対応

第4章で書いたスタイルが、NativeWindのクラスにどう対応するかを表で示します。

やりたいこと	StyleSheet	NativeWind クラス
画面いっぱい	<code>flex: 1</code>	<code>flex-1</code>
縦方向中央	<code>justifyContent: "center"</code>	<code>justify-center</code>
横方向中央	<code>alignItems: "center"</code>	<code>items-center</code>
横並び	<code>flexDirection: "row"</code>	<code>flex-row</code>
内側余白16	<code>padding: 16</code>	<code>p-4</code> (4 = 16px)
文字サイズ大	<code>fontSize: 24</code>	<code>text-2xl</code>
太字	<code>fontWeight: "bold"</code>	<code>font-bold</code>
背景白	<code>backgroundColor: "#fff"</code>	<code>bg-white</code>
角丸	<code>borderRadius: 8</code>	<code>rounded-lg</code>
文字色青	<code>color: "#1e40af"</code>	<code>text-blue-800</code>

数字の対応（重要）：Tailwindの余白やサイズの数字は「4倍するとピクセル」が基本です。`p-4` は `padding: 16`、`p-2` は `padding: 8`、`gap-3` は `gap: 12` です。慣れると暗算できます。色は `text-blue-600` のように「色名-濃さ（50～900）」で指定します。

4. 一覧画面をNativeWindで書き直す

第7・8章で `StyleSheet` で書いた一覧画面を、NativeWindで書き直してみます。`StyleSheet.create` のブロックがごっそり消えて、コードが短くなるのを体感してください。

▼このコードがやること（先に日本語で）：これまで `StyleSheet` で書いていた一覧画面を、すべて `className` で書き直した完成形です。注目してほしいのは、ファイル末尾にあった `StyleSheet.create({ ... })` の長いブロックがまるごと消え、見た目の指定が各部品 `className` に移っている点です。検索ボックス・カード・追加ボタンなどの見た目が、短いクラス名だけで表現されています。各クラスの意味やテンプレートリテラルでの色切り替えは、コード内コメントと直後の補足を見てください。

```
// app/index.tsx - NativeWind版 (主要部分のみ)

import { useState, useCallback, useMemo } from "react";
import { View, Text, FlatList, Pressable, TextInput, ActivityIndicator } from "react-native";
import { useRouter, useFocusEffect } from "expo-router";
import { fetchBooks } from "../lib/books";
import { Book } from "../types/book";

export default function HomeScreen() {
  const router = useRouter();
  const [books, setBooks] = useState<Book[]>([]);
  const [loading, setLoading] = useState(true);
  const [keyword, setKeyword] = useState("");

  const load = useCallback(async () => {
    try {
      setLoading(true);
      setBooks(await fetchBooks());
    } finally {
      setLoading(false);
    }
  }, []);

  useFocusEffect(useCallback(() => { load(); }, [load]));

  const filteredBooks = useMemo(() => {
    if (keyword.trim() === "") return books;
    const lower = keyword.toLowerCase();
    return books.filter(
      (b) => b.title.toLowerCase().includes(lower) ||
b.author.toLowerCase().includes(lower)
    );
  }, [books, keyword]);

  if (loading) {
    // className で中央寄せ。StyleSheet不要
    return (
      <View className="flex-1 justify-center items-center">
        <ActivityIndicator size="large" color="#1e40af" />
      </View>
    );
  }

  return (
    <View className="flex-1 bg-slate-50">
      { /* 検索ボックス */ }
      <View className="p-4 pb-0">
        <TextInput

```



```

        className="bg-white border border-slate-200 rounded-lg px-4 py-2.5 text-sm"
        placeholder="🔍 タイトルや著者で検索..."
        value={keyword}
        onChangeText={setKeyword}
    />
</View>

<FlatList
    data={filteredBooks}
    keyExtractor={({item}) => item.id}
    contentContainerClassName="p-4 gap-2.5" // contentContainerStyleのNativeWind版
    ListEmptyComponent={
        <Text className="text-center text-slate-400 mt-10 px-5">
            {keyword.trim() === "" ? "まだ本が登録されていません。" : "該当する本が見つかりませ
ん。"}
        </Text>
    }
    renderItem={({ item }) => (
        // カード。borderやrounded、paddingをすべてクラスで指定
        <Pressable
            className="bg-white rounded-xl p-3.5 border border-slate-200"
            onPress={() => router.push(`/books/${item.id}`)}
        >
            <Text className="text-base font-bold text-slate-800">{item.title}</Text>
            <Text className="text-xs text-slate-500 mt-0.5">著者: {item.author}</Text>
            {/* ステータスバッジ。色はstatusに応じてクラスを切り替える */}
            <View className={`self-start mt-2 px-2.5 py-0.5 rounded-full
${getStatusClass(item.status)}`>
                <Text className="text-xs font-semibold text-slate-700">{item.status}
            </Text>
            </View>
        </Pressable>
    )}
    />

    {/* 追加ボタン (FAB) */}
    <Pressable
        className="absolute right-5 bottom-8 w-14 h-14 rounded-full bg-blue-800
justify-center items-center shadow-lg"
        onPress={() => router.push("/new")}
    >
        <Text className="text-white text-3xl leading-8">+</Text>
    </Pressable>
</View>
);
}

// ステータスごとの背景色クラスを返す (第7章のgetStatusStyleのNativeWind版)
function getStatusClass(status: string) {
    if (status === "読了") return "bg-green-100";

```

```
if (status === "読書中") return "bg-blue-100";  
return "bg-amber-100";  
}
```

className の中で **`${ }`** を使う: `className={`... ${getStatusClass(item.status)}`}` のように、テンプレートリテラルで動的にクラスを足せます。ステータスによってバッジの背景色を変える、といった「条件で見た目を変える」処理に便利です。

contentContainerClassName: **FlatList** の内側のコンテナに対するクラス指定です。**StyleSheet** 版の **contentContainerStyle** に対応します。NativeWindは主要な部品でこうした **~ClassName** を用意しています。

StyleSheetと混在してOK: すべてを一度に書き換える必要はありません。NativeWindと **StyleSheet** は同じプロジェクトで共存できます。新しい画面はNativeWind、既存はそのまま、でも問題ありません。少しずつ慣れていきましょう。

5. アプリ全体の仕上げ

機能と見た目が整ったら、公開前に「アプリらしさ」を高める仕上げをしておくことで完成度が上がります。

5.1 アプリ名・アイコン・スプラッシュ画面

app.json（第1章で見た設定ファイル）で、アプリの基本情報を設定します。

▼このコードがやること（先に日本語で）: アプリの名前・アイコン・起動画面（スプラッシュ）などの基本情報を **app.json** にまとめて設定します。**name** はホーム画面に出るアプリ名、**icon** はアイコン画像、**splash** は起動時に一瞬出る画面の指定です。公開前の「アプリらしさ」を決める大事な設定なので、どの項目が何を表すかを各行のコメントで押さえておきましょう。

```

{
  "expo": {
    "name": "書籍管理", // ホーム画面に表示されるアプリ名
    "slug": "my-books-app", // プロジェクトの識別名（半角英数）
    "version": "1.0.0", // アプリのバージョン
    "icon": "./assets/icon.png", // アプリアイコン（1024x1024の正方形画像）
    "splash": { // 起動時に一瞬出るスプラッシュ画面
      "image": "./assets/splash.png", // スプラッシュ画像
      "backgroundColor": "#1e40af" // その背景色
    },
    "ios": { "supportsTablet": true }, // iPad対応
    "android": { "package": "com.example.mybooksapp" } // Androidの識別子（後で変更）
  }
}

```

アイコンとスプラッシュ画像: `assets` フォルダにExpoのデフォルト画像が入っています。まずはそのままでも公開できます。オリジナル画像に差し替えると一気に「自分のアプリ」感が出ます。アイコンは1024×1024ピクセルの正方形PNGを用意しましょう。第10章の公開前に整えるのがおすすめです。

5.2 ダークモード対応（発展・任意）

NativeWindは `dark:` というプレフィックスでダークモード（暗い配色）対応も簡単にできます。

```

// dark: を付けると、端末がダークモードのときだけそのクラスが適用される
<View className="bg-white dark:bg-slate-900">
  <Text className="text-slate-800 dark:text-slate-100">ダークモード対応のテキスト</Text>
</View>

```

任意の機能です: ダークモード対応は必須ではありません。「こんなこともできる」という紹介です。余裕があれば挑戦してみてください。

6. この章のまとめ

- スタイリングの選択肢（StyleSheet / NativeWind / Tamagui / UIキット）を比較し、目的別の選び方を理解した
- NativeWind を導入（インストール → `tailwind.config.js` → `global.css` → Babel/Metro設定 → `clear` 再起動）
- `style={...}` の代わりに `className="..."` でクラスを並べて見た目を指定する書き方を学んだ
- 一覧画面をNativeWindで書き直し、コードが短くなることを体感した

- `app.json` でアプリ名・アイコン・スプラッシュを設定し、公開の準備を整えた

次の章へ: アプリが完成しました！いよいよ第10章で、このアプリを **App Store** と **Google Play** に公開します。世界中の人があなたのアプリを使えるようになります。